

Agentic Research System

An AI-powered multi-agent research assistant that autonomously searches the web, extracts relevant information, generates comprehensive research reports, and critically evaluates its own outputs using specialized AI agents.

Overview

The Agentic Research System is a multi-agent application designed to automate the end-to-end research process. Unlike traditional chatbots that rely solely on pre-trained knowledge, this system actively gathers real-time information from the internet, analyzes multiple sources, synthesizes findings into structured reports, and performs a self-review to improve output quality.

The project demonstrates how Agentic AI architectures can be used to build systems capable of planning, tool usage, task decomposition, and iterative reasoning.

Problem Statement

Traditional conversational AI systems face several limitations:

- Limited access to recent information.
- Inability to verify facts using external sources.
- Lack of structured workflows for complex tasks.
- No mechanism to evaluate the quality of generated outputs.

Researchers and students often spend significant time collecting information, reading multiple articles, organizing findings, and reviewing their work.

This project addresses these challenges by creating an autonomous research pipeline powered by multiple specialized AI components.

Why Agentic AI?

Instead of assigning every responsibility to a single model, this system distributes tasks among specialized agents and tools.

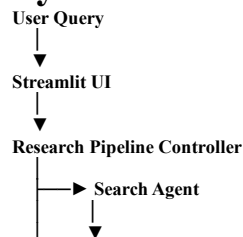
Benefits of the Agentic Approach:

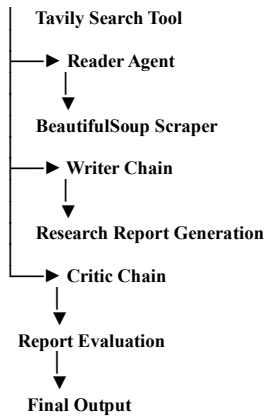
- Task specialization
- Improved reliability
- Better scalability
- Clear workflow separation
- Enhanced output quality through self-evaluation

The system mimics how human researchers work:

1. Search for information.
2. Read and analyze sources.
3. Write a report.
4. Review and critique work

System Architecture





Complete Workflow

Step 1: User Input

The user enters a research topic through the Streamlit interface.

Example:

"Quantum Computing Breakthroughs in 2025"

Step 2: Search Phase

The system invokes the web_search tool.

Responsibilities:

- Query Tavily Search API.
- Retrieve recent and reliable information.
- Return:
 - Titles
 - URLs
 - Snippets

Output:

Search Results → URLs + Summaries

Step 3: Reading Phase

URLs extracted from the search results are processed.

The scraper:

- Sends HTTP requests.
- Parses HTML using BeautifulSoup.
- Removes unnecessary elements:
 - Scripts
 - Navigation bars
 - Footers
 - Styles

The extracted text becomes the research context.

Step 4: Writing Phase

The Writer Chain receives:

- Search results
- Scraped content
- Original topic

It generates a structured report containing:

- Introduction
- Key Findings
- Conclusion
- Sources

Step 5: Critique Phase

The Critic Chain evaluates the generated report.

It provides:

- Numerical score
- Strengths
- Areas for improvement
- Final verdict

This introduces a feedback mechanism commonly used in advanced agentic systems.

Project Structure

Agentic-Research-System/

```

|
|— app_ui.py
|— pipeline.py
|— agents.py
|— tools.py
|— requirements.txt
|— .env
|— README.md

```

File Explanations

1. tools.py

Purpose:

Provides external capabilities to the agents.

Implemented Tools

Web Search Tool

Responsibilities:

- Connects to Tavily API.
- Searches the internet.
- Retrieves recent information.
- Returns formatted search results.

URL Scraping Tool

Responsibilities:

- Downloads webpage content.
- Cleans HTML.
- Extracts readable text.
- Provides detailed information for deeper analysis.

Role:

Agent → Tool → External Knowledge Source

2. agents.py

Purpose:

Defines the intelligence layer of the system.

Search Agent

Responsibilities:

- Uses the search tool.
- Retrieves factual information.
- Avoids relying solely on model memory.

Reader Agent

Responsibilities:

- Uses the scraping tool.
- Processes webpages.
- Extracts meaningful insights.

Writer Chain

Responsibilities:

- Converts research data into reports.
- Organizes information logically.
- Produces professional outputs.

Critic Chain

Responsibilities:

- Reviews generated reports.
- Identifies strengths.
- Detects weaknesses.
- Assigns quality scores.

3. pipeline.py

Purpose:

Acts as the orchestration engine.

Responsibilities:

- Coordinates all stages.
- Maintains system state.
- Passes outputs between components.

Pipeline sequence:

```
Search
↓
Scrape
↓
Write
↓
Critique
```

Without this file, the agents would operate independently without collaboration.

4. app_ui.py

Purpose:

Provides the user interface.

Features:

- Topic input field.
- Real-time pipeline visualization.

- Status indicators.
- Display of:
 - Sources
 - Reports
 - Critic feedback.
- Downloadable report generation.

The interface transforms backend intelligence into a usable application.

Technologies Used

Technology	Purpose
Python	Core programming language
LangChain	Agent and chain orchestration
Google Gemini 2.5 Flash	Large language model
Tavily API	Real-time web search
BeautifulSoup	HTML parsing
Requests	HTTP communication
Streamlit	Web interface
dotenv	Environment variable management
Rich	Enhanced console logging
Pydantic	Data validation support
Pandas	Data handling utilities
Tiktoken	Token processing
Tenacity	Retry mechanisms

Role of LangChain

LangChain enables:

- Tool integration.
- Agent creation.
- Prompt management.
- Output parsing.
- Modular workflow design.

It acts as the backbone connecting the LLM with external capabilities.

Role of Tavily Search

Tavily provides:

- Current information retrieval.
- Search result ranking.
- URL discovery.

Without Tavily, the system would be restricted to static model knowledge.

Role of BeautifulSoup

BeautifulSoup enables:

- Webpage parsing.
- HTML cleaning.
- Text extraction.

It transforms noisy webpages into structured research material.

Role of Streamlit

Streamlit provides:

- Interactive user experience.
- Visual pipeline monitoring.
- Research result presentation.

It converts the project into a deployable application.

Key Features

- Multi-agent architecture.
- Real-time web search.
- Automated webpage scraping.
- Structured report generation.
- Self-critiquing reports.
- Downloadable outputs.
- Interactive Streamlit dashboard.
- Modular and extensible design.

Installation and Setup

Clone Repository

```
git clone https://github.com/yourusername/Agentic-Research-System.git
cd Agentic-Research-System
```

Create Virtual Environment

```
python -m venv .venv
```

Activate environment:

Windows

```
.venv\Scripts\activate
```

macOS/Linux

```
source .venv/bin/activate
```

Install Dependencies

```
pip install -r requirements.txt
```

Configure Environment Variables

Create a .env file:

```
GOOGLE_API_KEY=your_api_key
```

```
TAVILY_API_KEY=your_api_key
```

Run Application

```
streamlit run app_ui.py
```

Challenges Faced

- Integrating multiple agents into a coherent workflow.
- Handling scraping failures and inaccessible webpages.
- Managing prompt consistency across agents.

- Preventing hallucinations through external knowledge grounding.
- Designing a UI that visualizes agent interactions effectively.

Future Enhancements

Potential improvements include:

- Memory-enabled agents.
- Parallel execution of agents.
- Citation verification mechanisms.
- PDF report export.
- Multi-modal document analysis.
- Human-in-the-loop approval stages.
- LangGraph-based workflow orchestration.
- Research history database.

Learning Outcomes

Building this project provided practical experience in:

- Agentic AI design patterns.
- Tool-augmented language models.
- LangChain workflows.
- Prompt engineering.
- Streamlit application development.
- Web scraping techniques.
- API integration.
- Software modularization.

Placement Value

This project demonstrates skills highly relevant to AI/ML and Software Engineering roles.

It showcases:

- System design thinking.
- Practical application of Generative AI.
- Multi-agent orchestration.
- Integration of external tools and APIs.
- End-to-end product development.

Unlike basic chatbot projects, this system highlights the ability to build intelligent applications capable of reasoning, information retrieval, content synthesis, and self-evaluation—making it a strong portfolio project for placements, internships, and research opportunities.

Conclusion

The Agentic Research System illustrates how specialized AI agents can collaborate to perform complex knowledge-intensive tasks. By combining web search, information extraction, report generation, and self-critique, the system moves beyond traditional conversational AI and demonstrates the potential of agentic workflows in solving real-world research problems.

It serves as both a practical productivity tool and a strong demonstration of modern AI engineering principles.